

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv("/content/clean_data.csv")
df.head(10)
```

	price	area	status	bhk	bathroom	age	location	builder
0	37.49	872	Ready to move	2	NaN	1.0	Sembakkam	MP Developers
1	93.54	1346	Under Construction	3	2.0	NaN	Selayur	DAC Promoters
2	151.00	2225	Under Construction	3	NaN	0.0	Mogappair	Casagrand Builder Private Limited
3	49.00	1028	Ready to move	2	2.0	3.0	Ambattur	Dugar Housing Builders
4	42.28	588	Under Construction	2	1.0	0.0	Pallavaram	Radiance Realty Developers India Ltd
5	188.00	2221	Under Construction	3	3.0	0.0	Virugambakkam	Traventure Homes Pvt Ltd
6	38.00	885	Under Construction	3	NaN	0.0	Thirumazhisai	Urbanrise
7	72.99	936	Ready to move	3	NaN	6.0	Moolakadai	Navin Housing Properties P LTD
8	125.00	2275	Ready to move	4	NaN	1.0	Ottiyambakkam	Jones foundation private limited
9	24.56	622	Under Construction	2	NaN	0.0	Perungalathur	Isha Homes

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df.columns
```

```
Index(['price', 'area', 'status', 'bhk', 'bathroom', 'age', 'location',
      'builder'],
      dtype='object')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2620 entries, 0 to 2619
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    price      2620 non-null    float64
1    area       2620 non-null    int64
2    status     2620 non-null    object
3    bhk        2620 non-null    int64
4    bathroom   1403 non-null    float64
5    age        1729 non-null    float64
6    location   2620 non-null    object
7    builder    2620 non-null    object
dtypes: float64(3), int64(2), object(3)
memory usage: 163.9+ KB
```

```
df.describe()
```

	price	area	bhk	bathroom	age
count	2620.000000	2620.000000	2620.000000	1403.000000	1729.000000
mean	93.834683	1282.925191	2.443893	2.359230	1.355119
std	113.609349	692.566319	0.811984	0.844951	2.102682
min	12.830000	300.000000	1.000000	1.000000	0.000000
25%	42.000000	877.000000	2.000000	2.000000	0.000000
50%	61.735000	1091.500000	2.000000	2.000000	0.000000
75%	90.000000	1471.250000	3.000000	3.000000	3.000000
max	1422.000000	6700.000000	8.000000	7.000000	32.000000

```
print(f"Total missing values in each columns \n {df.isnull().sum()}")
```

```
Total missing values in each columns
price      0
area       0
status     0
bhk        0
bathroom  1217
```

```
age          891
location     0
builder      0
dtype: int64
```

```
df.dtypes
```

```
      0
price  float64
area   int64
status object
bhk     int64
bathroom float64
age     float64
location object
builder object
```

```
dtype: object
```

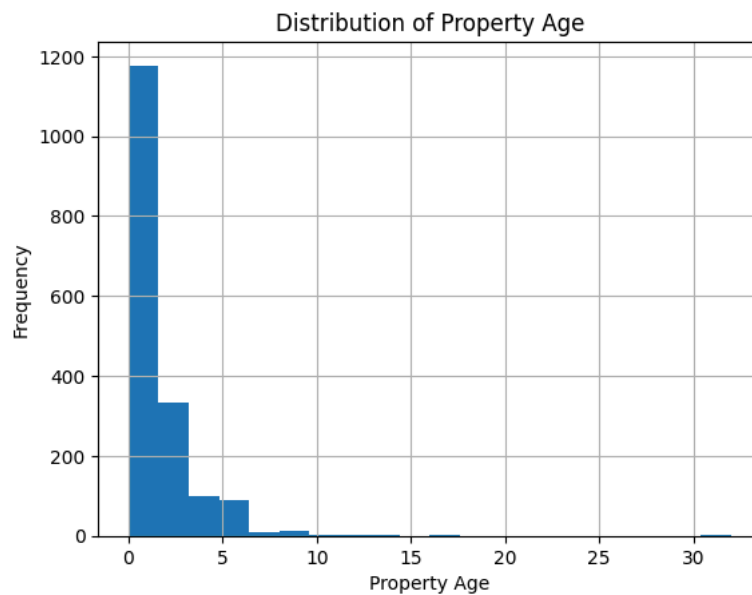
IMPUTATION FOR AGE

```
df['age'].value_counts()
```

```
      count
age
0.0      942
3.0      260
1.0      235
4.0      100
2.0       72
5.0       60
6.0       31
9.0       11
7.0        8
11.0        3
12.0         2
13.0         1
16.0         1
17.0         1
8.0          1
32.0         1
```

```
dtype: int64
```

```
df['age'].hist(bins=20)
plt.xlabel("Property Age")
plt.ylabel("Frequency")
plt.title("Distribution of Property Age")
plt.show()
```



```
df['age'].fillna(df['age'].median(), inplace=True)
```

/tmp/ipykernel_2387/2143548018.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through c. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]

```
df['age'].fillna(df['age'].median(), inplace=True)
```

```
print(df['age'].isnull().sum())
```

```
0
```

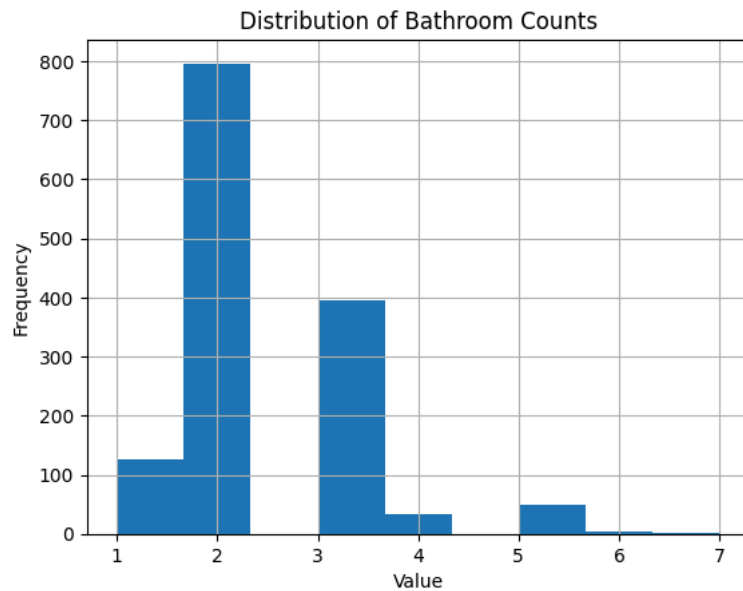
IMPUTATION FOR NUMBER OF BATHROOM

```
df['bathroom'].value_counts()
```

count	
bathroom	
2.0	796
3.0	395
1.0	125
5.0	49
4.0	33
6.0	4
7.0	1

```
dtype: int64
```

```
df['bathroom'].hist(bins=9)
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.title("Distribution of Bathroom Counts")
plt.show()
```



```
print(df['bathroom'].isnull().sum())
```

```
1217
```

```
df['bathroom'] = df.groupby('bhk')['bathroom'].transform(lambda x: x.fillna(x.median()))
```

CATEGORICAL COLUMNS

```
df['status'].value_counts()
```

count	
status	
Ready to move	1487
Under Construction	1133

```
dtype: int64
```

```
df['location'].value_counts().head(5)
```

count	
location	
Veppampattu	149
Pammal	139
Medavakkam	111
Sholinganallur	99
Perungudi	88

```
dtype: int64
```

```
df['builder'].value_counts().head(5)
```

count	
builder	
seller	484
MC Foundation	232
Appaswamy Real Estate	109
Propsource Realty Private Limited	79
Radiance Realty Developers	62

```
dtype: int64
```

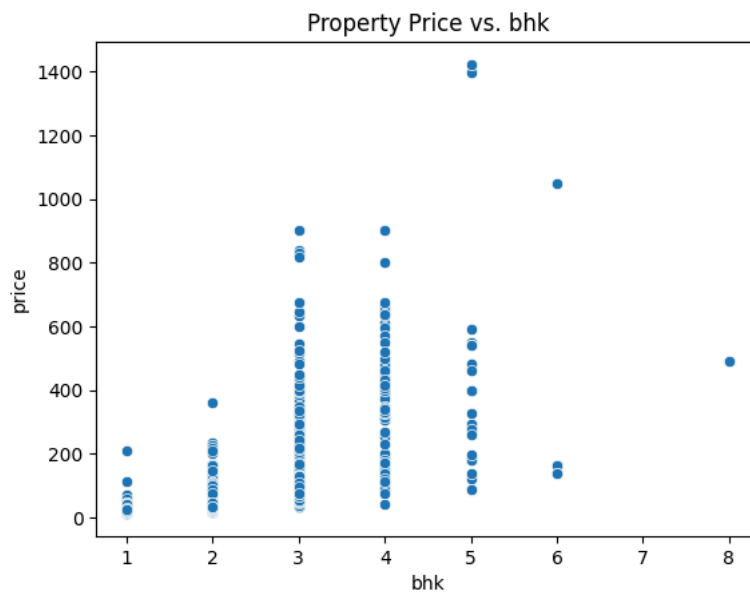
```
plt.title("Property Price vs. Area")  
sns.scatterplot(x='area', y='price', data=df)
```

<Axes: title={'center': 'Property Price vs. Area'}, xlabel='area', ylabel='price'>



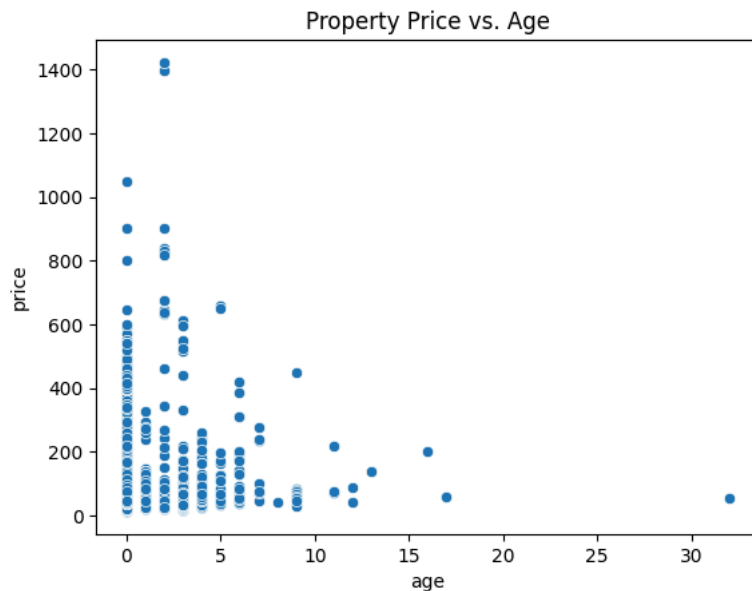
```
plt.title("Property Price vs. bhk")  
sns.scatterplot(x='bhk', y='price', data=df)
```

<Axes: title={'center': 'Property Price vs. bhk'}, xlabel='bhk', ylabel='price'>



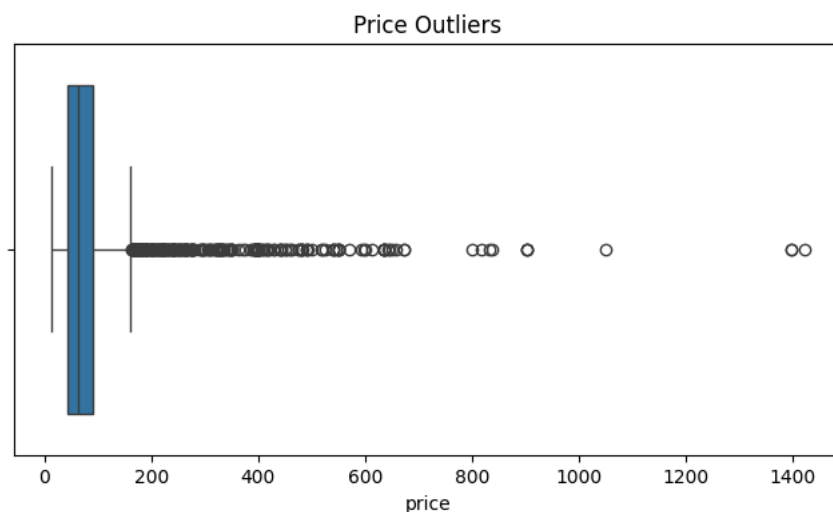
```
plt.title("Property Price vs. Age")  
sns.scatterplot(x='age', y='price', data=df)
```

```
<Axes: title={'center': 'Property Price vs. Age'}, xlabel='age', ylabel='price'>
```

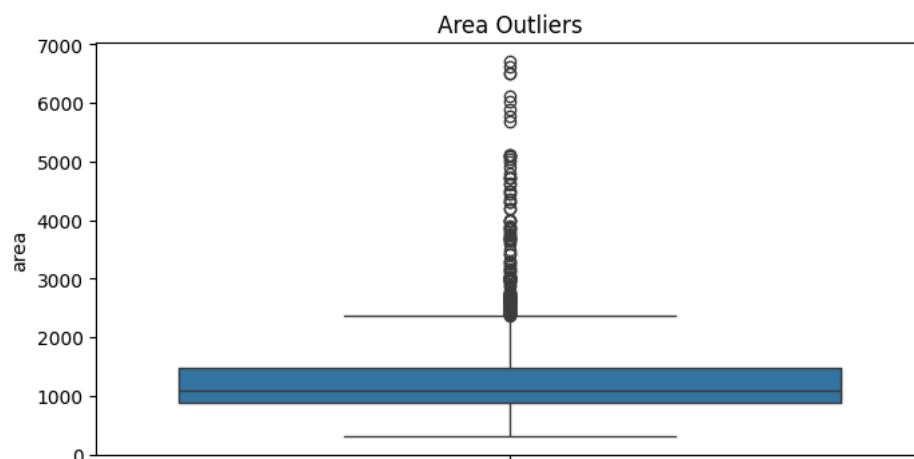


OUTLIER DETECTION

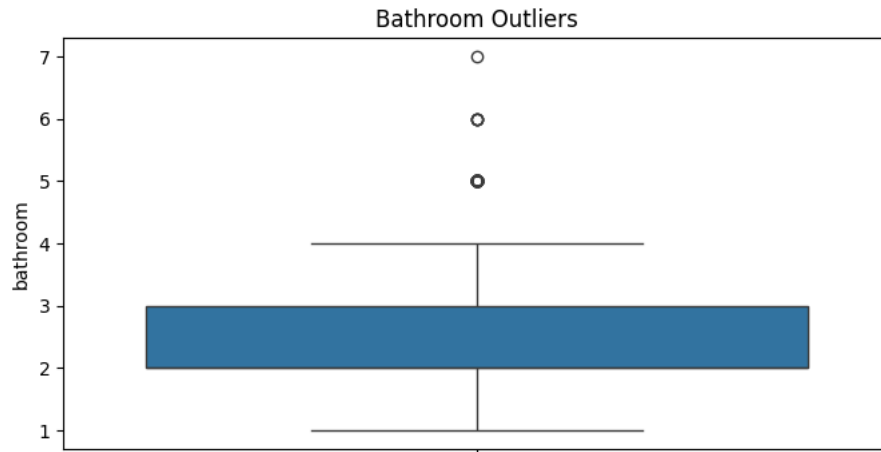
```
plt.figure(figsize=(8,4))
sns.boxplot(x=df['price'])
plt.title("Price Outliers")
plt.show()
```



```
plt.figure(figsize=(8,4))
sns.boxplot(y=df['area'])
plt.title("Area Outliers")
plt.show()
```



```
plt.figure(figsize=(8,4))
sns.boxplot(y=df['bathroom'])
plt.title("Bathroom Outliers")
plt.show()
```



```
cols = ['price', 'area', 'bathroom', 'age', 'bhk']

for col in cols:

    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)

    IQR = Q3 - Q1

    l = Q1 - 1.5 * IQR
    u = Q3 + 1.5 * IQR

    df = df[(df[col] >= l) & (df[col] <= u)]

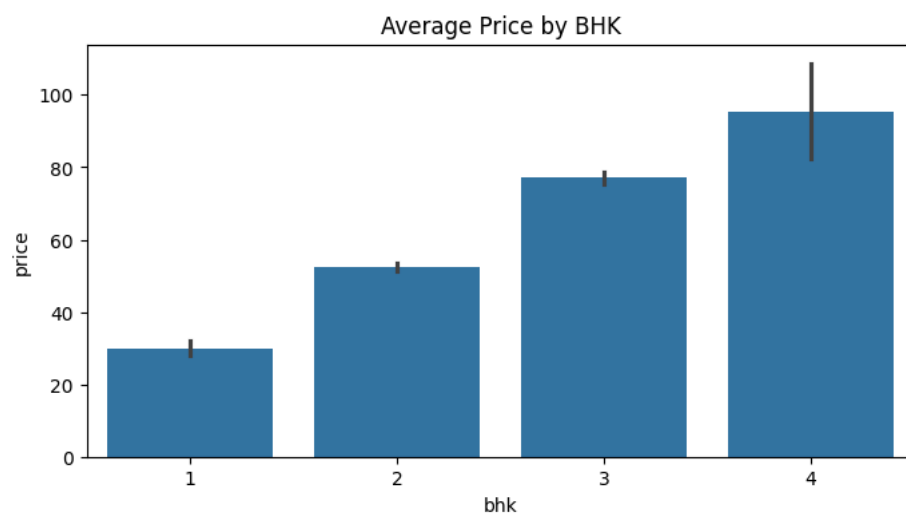
print("Shape after outlier removal :", df.shape)
```

Shape after outlier removal : (1526, 8)

```
plt.figure(figsize=(8,4))

sns.barplot(
    x='bhk',
    y='price',
    data=df
)

plt.title("Average Price by BHK")
plt.show()
```



```
plt.figure(figsize=(8,4))

sns.barplot(
    x='status',
    y='price',
```

```
data=df
)

plt.xticks(rotation=20)
plt.title("Average Price by Status")
plt.show()
```

